

Algorithms - Final exam

اتحاد طلبة هندسة
الحاسوب والشبكات



computer & networks
engineers

TOGETHER WE REACH



عضو الجمعية :
عمر خريسات

facebook.com/groups/CNE.FET/

<http://cne-fet.com>

Fill in the answer sheet with the most appropriate answers:

1	2	3	4	5
B	B	A	D	A
6	7	8	9	10
B	A	C	B	A
11	12	13	14	15
C	B	A	D	A
16	17	18	19	20
D	C	B	A	E



- 9) (time complexity) how many time statement "good luck" will be printed

```
for (k=1; k<=n; k++)
    cout<<"good luck";
for (j = 1; j <= n*n*n; j++)
    cout<<"good luck";
```

- A. $n/3$ B. $\log_3 n + n^3$ C. $n + n^3$ D. $n/3 + n^3$

- 10) Which is the correct increasing ordering of the following runtime complexities:

$O(n)$, $O(2^n)$, $O(n^3)$, $O(n \log n)$, $O(n^2)$, $O(\log n)$, $O(n!)$

- A. $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(n!)$
 B. $O(\log n)$, $O(n \log n)$, $O(n)$, $O(n^2)$, $O(2^n)$, $O(n^3)$, $O(n!)$
 C. $O(n!)$, $O(2^n)$, $O(n^3)$, $O(n^2)$, $O(n \log n)$, $O(n)$, $O(\log n)$
 D. $O(\log n)$, $O(n \log n)$, $O(n)$, $O(n^2)$, $O(n^3)$, $O(n!)$, $O(2^n)$

- 11) Consider the following recursive algorithm. Analyze the code: Algorithm

```
MysteryPrint(A[1..n]);
begin
    if n = 1 then
        print A[1]
    else begin
        callMysteryPrint(A[1..n/2]);
        callMysteryPrint(A[2..n])
    end
end
```

- end:
 A. $T(n) = T(n) + T(n/2)$ C. $T(n) = T(n-2) + T(n-1)$
 B. $T(n) = 2T(n) + T(n-1)$ D. $T(n) = T(n-1) + T(n/2)$

- 12) (time complexity) how many time statement "AMMAN" will be printed

```
for (j = 1; j <= n; j++)
    cout<<"AMMAN";
for (j = 1; j <= n; j++)
{
    j = 3*j;
    cout<<"AMMAN";
}
```

- A. $n/3 + n$ B. $\log_3 n + n$ C. $n + n^3$ D. $n + n^3$
 13) $F(n) = n^2 + 5n$?
 A. $O(n^2)$ B. $O(n \log n)$ C. $O(\log n)$ D. $O(n)$

- 14) The relationship between two complexity functions 2^n and 3^n
 A. $2^n \in \Omega(3^n)$ B. $2^n \in \Theta(3^n)$ C. $3^n \in O(2^n)$ D. $2^n \in O(3^n)$





- 1) The time complexity of an algorithm is measured by:
A. Counting microseconds
☒ B. Counting the number of key operations
C. Counting the number of statements
D. Counting the kilobytes of algorithm
- 2) The worst case complexity of Binary search algorithm is
A. $O(n)$ ☒ B. $O(\log n)$ C. $O(n^2)$ D. $O(n \log n)$
- 3) The Average case occur in linear search algorithm
☒ A. When Item is somewhere in the middle of the array
B. When Item is not in the array at all
C. When Item is the last element in the array
D. When Item is the last element in the array or is not there at all
- 4) The complexity of merge sort algorithm is
A. $O(n)$ B. $O(\log n)$ C. $O(n^2)$ ☒ D. $O(n \log n)$
- 5) The worst case complexity of linear search algorithm is
☒ A. $O(n)$ B. $O(\log n)$ C. $O(n^2)$ D. $O(n \log n)$
- 6) Two main measures for the efficiency of an algorithm are
☒ A. Processor and memory
☒ B. Complexity and capacity
C. Time and space
D. Data and space
- 7) Big-oh notation is a method that is used to express the of the running time of an algorithm.
☒ A. Lower bound B. Upper bound C. Lower and upper bound
D. none
- 8) A particular function takes a positive integer parameter n and performs exactly $3n+2n \log(n)+50 \log(n)+1000$ steps before returning. What is its runtime complexity?
A. $\Omega[\log(n)]$ b. $\Omega[n \log(n)]$ ☒ C. $\Omega[n]$ D. a, b & c correct

15) The relationship between two complexity functions $\log_2 n$ and $\log_3 n$.

- A. $\log_2 n \in \Theta(\log_3 n)$ B. $\log_2 n \in O(\log_3 n)$ C. $\log_2 n \in \Omega(\log_3 n)$
 D. $\log_2 n \in O(\log_2 n)$

16) (time complexity) how many time statement "good luck" will be printed

```
for (j = 1; j <= n; j++) {
    cout << "good luck";
    for (j = 1; j <= n; j++)
        for (j = 1; j <= n; j++)
            cout << "good luck";
}
```

A. n

B. $n + n^2$

C. $\log_2 n + n$

D. $n + n^3$

17) Consider the following recursive algorithm. Analyze the code: Algorithm
 MysteryPrint(A[1..n]);

begin

if $n = 1$ then

print A[1]

else begin

callMysteryPrint(A[1..n/2]);

callMysteryPrint(A[1..n/2]);

end

end;

A. $T(n) = T(n) + T(n/2)$

B. $T(n) = T(n-2) + T(n-1)$

C. $T(n) = 2T(n/2)$

D. $T(n) = T(n-1) + T(n/2)$



18) Big-omega notation is a method that is used to express the of the running time of an algorithm.

- a. Lower bound b. Upper bound c. Lower and upper bound d. none

19) Which statement is wrong according to Fibonacci sequence algorithms:

- a. Iterative algorithm is efficient than recursive algorithm
 b. The recursive algorithm has an exponential time complexity,
 c. The iterative algorithm has a linear time complexity $\Theta(n)$
 d. The recursive algorithm has a linear time complexity $\Theta(n)$

20) Which statement is wrong according to divide and conquer (D&Q) design method:

- a. We can implement binary search using D&Q.
 b. We can implement merge sort using D&Q.
 c. It is a bottom-up approach
 d. We can implement recursive algorithm for Fibonacci using D&Q.

- 1) The time complexity of an algorithm is measured by:
 - A. Counting microseconds
 - ☒ B. Counting the number of key operations
 - C. Counting the number of statements
 - D. Counting the kilobytes of algorithm
- 2) The worst case complexity of Binary search algorithm is
 - A. $O(n)$
 - ☒ B. $O(\log n)$
 - C. $O(n^2)$
 - D. $O(n \log n)$
- 3) The Average case occur in linear search algorithm
 - ☒ A. When Item is somewhere in the middle of the array
 - B. When Item is not in the array at all
 - C. When Item is the last element in the array
 - D. When Item is the last element in the array or is not there at all
- 4) The complexity of merge sort algorithm is
 - A. $O(n)$
 - B. $O(\log n)$
 - C. $O(n^2)$
 - ☒ D. $O(n \log n)$
- 5) The worst case complexity of linear search algorithm is
 - ☒ A. $O(n)$
 - B. $O(\log n)$
 - C. $O(n^2)$
 - D. $O(n \log n)$
- 6) Two main measures for the efficiency of an algorithm are
 - A. Processor and memory
 - ☒ B. Complexity and capacity
 - C. Time and space
 - D. Data and space
- 7) Big-oh notation is a method that is used to express the of the running time of an algorithm.
 - ☒ A. Lower bound
 - B. Upper bound
 - C. Lower and upper bound
 - D. none
- 8) A particular function takes a positive integer parameter n and performs exactly $3n + 2n \log(n) + 50 \log(n) + 1000$ steps before returning. What is its runtime complexity?
 - A. $\Omega[\log(n)]$
 - b. $\Omega[n \log(n)]$
 - ☒ C. $\Omega[n]$
 - D. a, b & c correct

